

**INSTITUTO
FEDERAL**
Piauí

Campus
Floriano

Aula 1.

Fundamentos da Web

Prof. Ronaldo Borges

Como Tudo Começou

O Problema

- **Onde:** CERN (Organização Europeia para a Pesquisa Nuclear), 1990.
- **O Criador:** Tim Berners-Lee.
- **Dificuldade:** Cientistas precisavam de um mecanismo simples para disseminar e conectar documentos de pesquisa isolados em diferentes computadores.



A Solução

WWW: Criação de um sistema de documentos interligados acessíveis via internet. Nascia a base da World Wide Web.



A Web foi criada "ao redor" do Hipertexto



A Web não é apenas uma rede de computadores, mas fundamentalmente uma rede de **hipertextos** navegáveis.

Arquitetura Cliente-Servidor

Cliente (Front-end)



- **Função:** Inicia a comunicação. Solicita recursos (dados/páginas), recebe a resposta e renderiza a interface para o usuário.
- **Exemplos:** Chrome, Firefox, Apps Mobile.

Requisição
(Request)



Resposta
(Response)



Servidor (Back-end)



- **Função:** Escuta passivamente as solicitações, processa regras de negócio, consulta o banco de dados e devolve o hipertexto/dados solicitados.
- **Exemplos:** Apache, Nginx, aplicações Node, PHP, Java.

Protocolo HTTP (A Linguagem da Web)



- **O que é?** HyperText Transfer Protocol. O conjunto estrito de regras que padroniza a comunicação na web.
- **Como funciona?** Baseia-se exclusivamente no ciclo síncrono de Requisição e Resposta.
- **Característica Chave:** É um protocolo Stateless (sem estado). Cada requisição é tratada de forma independente; o servidor não guarda memória das requisições anteriores por padrão.
- **Exemplo Real:** Ao digitar `http://www.ifpi.edu.br` no navegador e apertar Enter, você dispara uma Requisição HTTP.

Dissecando uma Requisição HTTP (Request)

Método + Caminho.
Ação desejada e onde o recurso está localizado.

```
GET /cursos/k02 HTTP/1.1
Host: www.k19.com.br
Accept: text/html
User-Agent: Mozilla/5.0
```

Versão do Protocolo.

Cabeçalhos (Headers).
Metadados cruciais. O cliente avisa quem ele é (User-Agent) e que formato de arquivo espera receber (Accept: text/html).

Métodos HTTP: Os "Verbos" da Web

O método define a intenção do cliente.



GET (Buscar/Ler)

Solicita a representação de um recurso.

Ex: Carregar a página inicial de um portal de notícias.



POST (Criar/Enviar)

Submete dados ao servidor para processamento ou criação.

Ex: Enviar um formulário de login.



PUT (Atualizar)

Substitui todas as atuais representações do recurso de destino.

Ex: Atualizar dados inteiros do seu perfil.



DELETE (Remover)

Apaga um recurso específico.

Ex: Deletar uma foto da galeria.

Dissecando uma Resposta HTTP (Response)

Status Code. O resultado da operação (Neste caso, Sucesso!).

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2024 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 131
```

Headers. O servidor instrui o navegador sobre como interpretar o conteúdo.

```
<html>
  <body>Olá, Web!</body>
</html>
```

Corpo (Body). O conteúdo final entregue (o hipertexto propriamente dito que será desenhado na tela).

Status Codes: Entendendo o Resultado

1xx (Informativo): A requisição foi recebida e o processo continua (Tráfego normal).

2xx (Sucesso): A ação foi recebida, compreendida e aceita com sucesso (Tudo certo).

3xx (Redirecionamento): O cliente precisa tomar ações adicionais (Ex: O endereço da página mudou).

4xx (Erro do Cliente): A requisição possui sintaxe incorreta ou não pode ser processada (O erro foi de quem pediu).

5xx (Erro do Servidor): O servidor falhou ao processar uma requisição aparentemente válida (O erro é na infraestrutura/código de quem entrega).

Os Status Codes Mais Famosos na Prática



200 OK

O caminho feliz! O servidor encontrou o recurso solicitado e o anexou perfeitamente no corpo da resposta.



404 Not Found

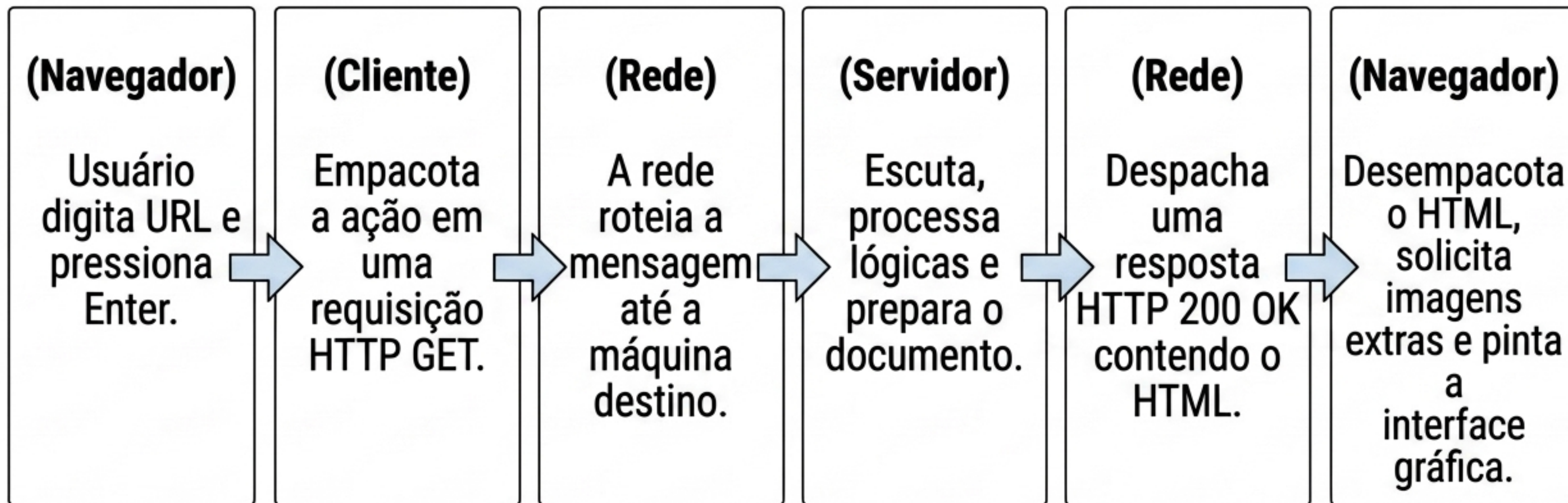
Clássico! O cliente pediu uma URL que não existe no servidor (ex: você digitou um endereço errado na barra do navegador).



500 Internal Server Error

O pedido estava correto, mas a aplicação do lado do servidor "quebrou" ao tentar processar (ex: banco de dados caiu, erro de lógica no backend).

O Ciclo de Vida Completo na Web



Toda essa complexidade de engenharia ocorre de forma invisível em frações de segundos, sustentando a web como a conhecemos.

Referências

- CAELUM. Desenvolvimento Web com HTML, CSS e JavaScript. Apostila. São Paulo: Caelum, 2022.
- CORDEIRO, J. J. R. HTML5 e CSS3: Domine a web do futuro. São Paulo: Casa do Código, 2013.
- FLANAGAN, D. JavaScript: The Definitive Guide. 7. ed. Sebastopol: O'Reilly Media, 2020.
- K19. K02 - Desenvolvimento Web com HTML, CSS e JavaScript. São Paulo: K19 Treinamentos, 2013.
- SILVA, M. S. HTML5: A linguagem de marcação que revolucionou a web. 2. ed. São Paulo: Novatec Editora, 2024.